Threat model (short)

Assets to protect:

- Host OS.
- Host filesystem,
- user credentials,

Adversary capabilities:

- Remote web attacker able to deliver drive-by exploits,
- crafted downloads (malicious binaries, office macros, PDFs),
- cross-site attacks;
- may attempt to escape browser sandbox and access host files.

Assumptions / limits:

- Host kernel and hypervisor are up-to-date,
- physical access is not considered,
- social-engineering (user intentionally copying malware), (Inside Threat)
- we focus browser-originated compromise and downloaded-file sanitization.

Project goal (primary + secondary)

Primary goal:

- runs an untrusted browser session inside a local micro-VM,
- intercepts downloads into the VM,
- sanitizes/inspects them,
- only exposes a policy-limited safe artifact to the host.

Secondary goals:

- Measure economic trade-offs (startup latency, memory/CPU overhead),
- produce a simple UX wrapper,
- evaluate sanitizer effectiveness against real malware (in a controlled lab).

Success criteria

Downloads are captured into the micro-VM disk;

automatic sanitizer runs and produces either (a) safe copy + metadata, or (b) blocked + report.

Host never receives raw downloaded bytes until sanitized.

Measured overhead documented:

TESTING MESUREMENT

VM startup time ≤ 1.5s (target for micro-VM),

memory overhead per session recorded,

page load latency compared to native baseline.

A short evaluation (10–20 test pages/files) demonstrating blocked or sanitized malicious samples in the VM.

Functional correctness:

Verified that downloads always remain in VM until sanitizer completes;

host only receives sanitized outputs or is denied.

Test vectors:

benign documents,

password-protected ZIPs,

malicious PE files,

malicious PDFs (controlled lab).

TESTING SANITIZE ENGINE

Security measurement:

Demonstrate containment by running known exploit PoCs in the VM and showing no host compromise (use safe, documented PoCs only).

Document residual risks (e.g., hypervisor escape).

Performance metrics:

VM startup time,

memory & CPU per session,

average page load time vs native Chrome,

time-to-sanitize-per-file. Report averages ± stdev over repeated runs.

Concrete architecture diagram (ASCII)

Host Operating system

Host Controller(Go/Python)

<--vsock--> Micro-VM Manager (start/stop micro-VM)

policy decisions

audit/logging

VSOCK / Vsock Proxy

Micro-VM Instance (rootfs) |

(control channel)

Chromium

download dir (isolated)

sanitizer (clamav, file-type tools, parsers)

transfer agent (vm->host)

Host-side transfer agent

receives sanitized files

Micro-VM layer

Firecracker microVM (lightweight): https://github.com/firecracker-microvm/firecracker

Install:

sudo apt install -y jq build-essential pkg-config libseccomp-dev

(build per README) or use Ubuntu packages where available.

Create kernel + minimal rootfs (see Firecracker docs).

You can generate a rootfs from a Debian container:

`docker run --rm --privileged -v $(pwd):/out debian:buster sh -c 'apt-get update && apt-get install -y chromium Xvfb ... && tar -C / -c . > /out/rootfs.tar'`

(adapt to produce minimal rootfs).

Controller language

Go or Python (Go recommended for reliable concurrency):

Go libs:

`net/http` (for Firecracker API),

`github.com/jackc/pgx` (if logging to Postgres),
`github.com/containers/gvisor-tap-vsock` (if experimenting with vsock).

Use `firecracker-go-sdk` for convenience.

Browser & rendering

Chromium inside micro-VM. Use Puppeteer (Node) or Selenium inside the VM to manage browser sessions if needed.

For prototypes, headless + periodic screenshots is acceptable.

`apt install -y chromium` and run `chromium --no-sandbox --remote-debugging-port=9222`.

Communication / I/O

virtio-vsock (vsock) or a small socket proxy (host-side) to pass control commands and file-transfer requests. Experiment with `gVisor` or `vsock-proxy` implementations.

Sanitizer & file-inspection

`libmagic` (file type), `ClamAV` (antivirus), `pandoc` or LibreOffice headless (for conversion DOCX->PDF), `pdf-redact-tools` or `qpdf` (for PDF processing), `oletools` (for office macro inspection). Example inside VM: `apt install -y clamav libfile-magic-perl qpdf libreoffice`.

Example sanitize step:

`file --mime-type sample.docx && clamscan --no-summary sample.docx && libreoffice --headless --convert-to pdf sample.docx`.

Host/UI

Small Electron or web UI wrapper (Node + Electron or simple Flask web UI). The UI talks to host controller via REST or websocket. Use NoVNC or simple screenshot polling for display in early prototype.

Logging & audit

Simple SQLite or PostgreSQL; structured JSON logs for each session and transfer.